# Cpp Introduction

André Cerqueira

FCUP

2024

Este trabalho foi financiado por: Projeto 10110190 – EUROCC2, com apoio financeiro da FCT/MCTES através de fundos nacionais (PIDDAC).

# Structure of a program

```
/*
 first cpp code
*/
#include <cmath>
#include <iostream>
using namespace std;

void printName() { cout << "Sycl
↪  Learners" << endl; }

// Execution starts at main()
int main() {
  cout << "Hello: " << endl;
  printName();
  printName();
  return 0;
}
```

- Comments:
  - "Anywhere in the Code"
  - "//"
  - Multi Line: "/*" and "*/"
- Compiler Directives
  - #includes tell compiler what other library functions you plan on using
- main() function
  - Starting point of execution for the program
  - Statements end with a semicolon (;)
  - Ends with 'return 0;' statement

# Fundamental Data Types

- In C++ constants and variables can be of different types and sizes

| Type | Description | Range |
|---|---|---|
| char | Character or Small Integer | $-128$ to 127 |
| short int | Short Integer | $-32768$ to 32767 |
| int | Integer | $-2147483648$ to 2147483647 |
| bool | Boolean Value | true or false |
| float | Floating Point Number | $+/-3.4e+/-38$ ( around 7 digits) |
| double | Double precision floating point number. | $+/-1.7e+/-308$ ( around 15 digits) |

Table: Fundamental Data Types

# Arithmetic Operators

- Addition, subtraction, multiplication work as expected for both integer and floating point types
- Division works 'differently' for integer vs. doubles/floats

| Operator | Name | Example |
|---|---|---|
| + | Addition | 2+5 |
| − | Subtraction | 41 - 32 |
| ∗ | Multiplication | 4.23 * 3.1e-2 |
| / | Division | 10/3 or 10.0/3 |
| % | Modulus | 17%5 |

Table: Arithmetic Operators

# Input/Output

- I/O is placed in temporary buffers/streams by the OS/C++ libraries.
- **cin** goes and gets data from the input stream (skipping over preceding whitespace then stopping at following whitespace)
- **cout** puts data into the output stream for display by the OS (a flush forces the OS to display the contents immediately)

```
#include <iostream>
using namespace std;
int main() {
  cout << "It was the" << endl;
  cout << "best of times.";
}
```

```
#include <iostream>
using namespace std;
int main() {
  int x;
  cin >> x;
  return 0;
}
```

# C++ Output

- Include <iostream> (not iostream.h)
- Add using namespace std; at top of file
- 'cout' requires appropriate use of separators << between consecutive values or different types of values
- 'cout' does not add spaces between consecutive values; you must do so explicitly

```
#include <iostream>
using namespace std;
int main(int argc, char
↪  *argv[]) {
  int x = 5;
  char c = 'Y';
  double y = 4.5;
  cout << "Hello world" <<
  ↪  endl;
  cout << "x = " << x;
  cout << " c = " << c <<
  ↪  "\ny is " << y <<
  ↪  endl;
  return 0;
}
```

# C++ Input

- cin (character input) object used to accept input from the user and write the value into a variable
- Every '>>' will skip over any leading whitespace looking for text it can convert to the variable form, then stop at the trailing whitespace

```
#include <iostream>
using namespace std;
int main(int argc, char
↪  *argv[]) {
  int x;
  char c;
  double y;
  cout << "Enter an
  ↪  integerand double
  ↪  separated by spaces:
  ↪  "
       << endl;
  cin >> x >> c >> y;
  cout << "x = " << x << "
  ↪  c = ";
  cout << "y is " << y <<
  ↪  endl;
  return 0;
}
```

# Pointers

- A pointer is just a memory location.
- A memory location is simply an integer value, that we interpret as an address memory (**0xfe4a10c5**)

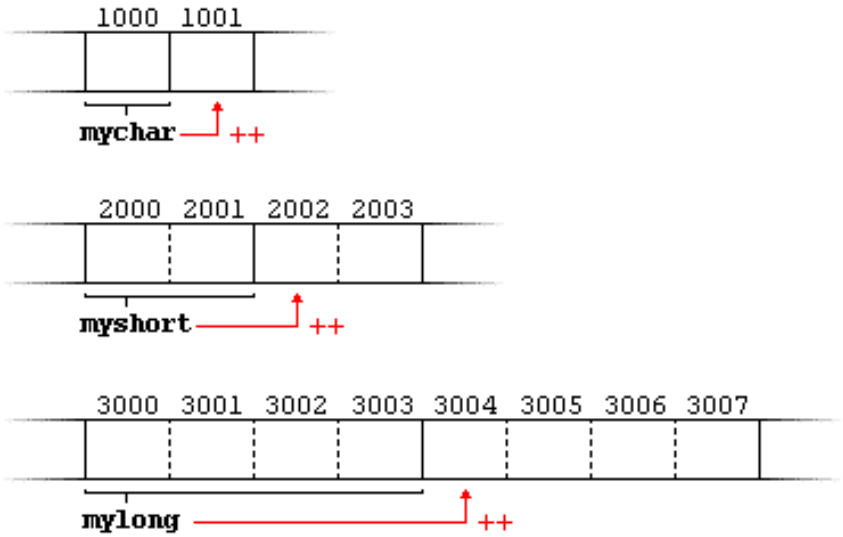
```
#include <iostream>
using namespace std;

int main ()
{
  int firstvalue, secondvalue;
  int * mypointer;

  mypointer = &firstvalue;
  *mypointer = 10;
  mypointer = &secondvalue;
  *mypointer = 20;
  cout << "firstvalue is " << firstvalue << '\n';
  cout << "secondvalue is " << secondvalue << '\n';
  return 0;
}
```

# Pointer Arithmetic

- $*p++$: same as *(p++): increment pointer, and dereference incremented address
- $*++p$: same as *(++p): increment pointer, and dereference incremented address
- $++*p$: same as ++(*p): dereference pointer, and increment the value it points to
- $(*p)++$: dereference pointer, and post-increment the value it points to



# Lambda Calculus

- C++ Lambda expression allows us to define anonymous function objects (functors) which can either be used inline or passed as an argument.
- Lambda expression was introduced in C++11 for creating anonymous functors in a more convenient and concise way.
- Main way of launching kernels in **SYCL**

```
auto greet = []() {
  // lambda function body
};

void greet() {
  // function body
}
```

```
int main() {
  auto greet = []() {
    cout << "Hello World!";
  };

  greet();
  return 0;
}
```

# Lambda Calculus

The basic syntax is:
[ capture-list ] ( params ) -> ret {body}
where:

- **capture-list** is a comma-separated list of captures. There are also shorthands that apply to all in-scope automatic variables: [=] is used to capture all automatic variables used in the body by value and the current object by reference, [&] is used to capture all automatic variables used in the body as well as the current object by reference, and [] captures nothing.
- **params** is the list of function parameters
- **ret** is the return type.
- **body** is the function body

# Lambda Calculus

```
int i = 1, j = 10, k = 100, l = 1000;
auto lambda = [i, &j] (int k0, int &l0) -> int {
 j = 2* j;
 k0 = 2* k0;
 l0 = 2* l0;
 return i + j + k0 + l0;
};
```